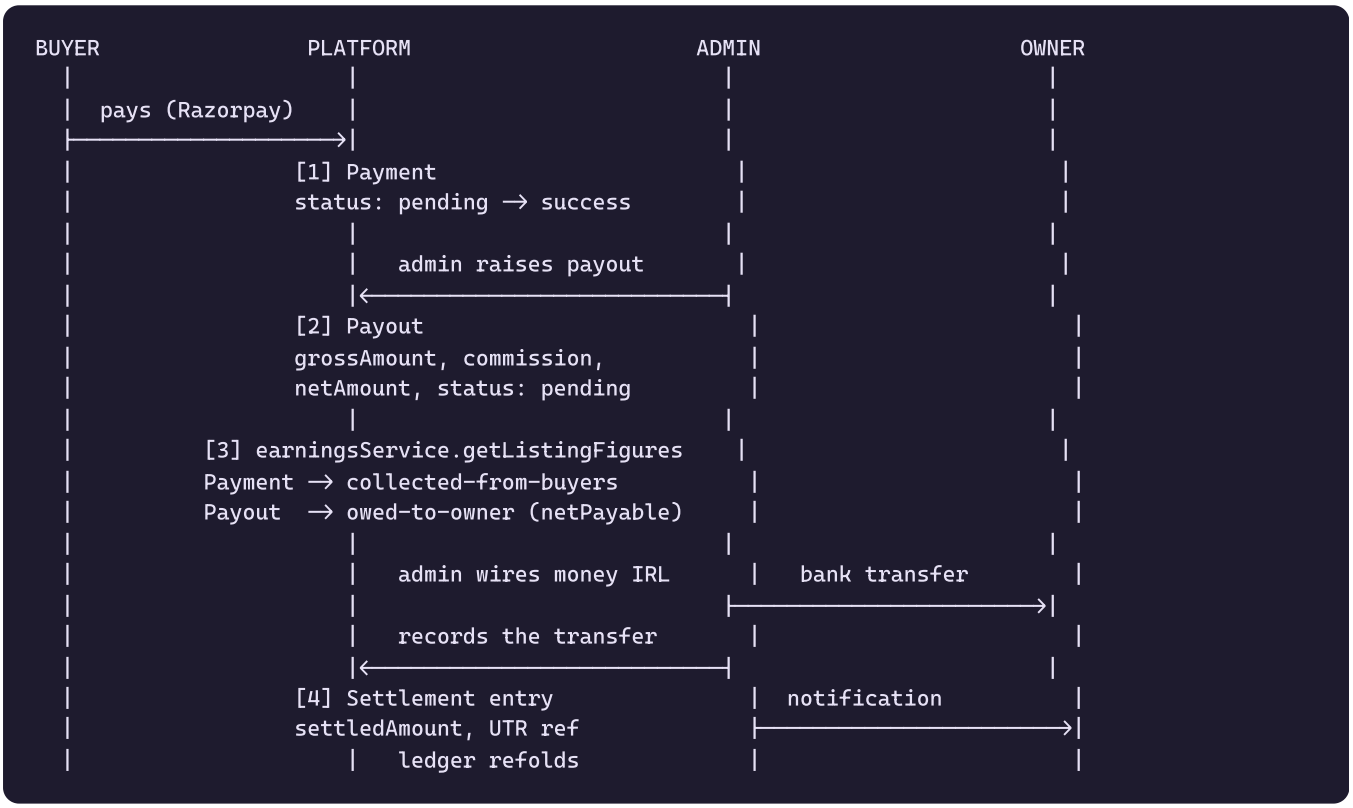
The Settlement Flow

How money moves from a buyer to a listing owner, end to end

Platform payments & settlement · Technical overview

> Settlement is the **manual, append-only ledger** that records real bank transfers from the platform to a listing owner (event organizer or venue owner). It is built on four record types, and the cardinal rule across the whole system is: **money is summed verbatim from recorded fields, never recomputed a second way.**

The flow at a glance



Stage 1 Money In (Payment)

When a buyer pays, the payment service runs a two-step Razorpay flow:

- **Initiate** — writes a `Payment` with `status: pending`. The billing math runs first: `subtotal = price × qty`, `platformFee = subtotal × fee%`, `gstAmount = platformFee × 0.18` (GST is 18% *of the platform fee*, not the ticket), `totalAmount = subtotal + fee + gst − discount`.
- **Verify** — validates the gateway signature, flips `status: success`, stamps `paidAt`.
- **Refund** — sets `status: refunded`.

An event-ticket payment carries `referenceModel: 'Event'`; a venue payment carries `referenceModel: 'Booking'`. `success` is the single meaning of “collected.”

Stage 2 Payout Creation (Payout)

A `Payout` is created **manually by an admin** — there is no scheduled job, nothing creates it automatically. It computes:

- `platformCommission = grossAmount × commission%`
- `netAmount = grossAmount − platformCommission`

It **fails closed on bank details**: it loads the owner, prefers the account the listing named (falling back to the owner default), and throws if there are no valid bank details on file. The payout stores a **snapshot** of those details (not a reference, so deleting the account later cannot rewrite history), with `status: pending`.

> **Key point.** `netPayable` across the entire settlement system comes **only from** `Payout.netAmount`. A listing with collected payments but no payout has `netPayable = 0` and nothing to settle. That is why "Owed to Owner" can read ₹0.00 while "Gross collected" shows a real amount — payments exist, but no payout has been raised yet.

Stage 3 Earnings Aggregation

A single service aggregates the figures. Two sums that never cross:

Side	Source	Fields
Collected from buyers	Payment (success / refunded)	grossCollected, platformFeeCollected, gstRetained, refundedTotal
Owed to owner	Payout	ownerGross, platformCommission, netPayable

Plus activity (tickets / bookings sold, confirmed, cancelled, last payment) and a representative payout (the most recent one's status and net amount).

Stage 4 The Ledger Fold

The recorded settlement rows are folded into three **derived** figures:

- `settledToDate` = sum of effective `settledAmount`, skipping both reversal rows and any row that was reversed — so a reversed pair nets to zero by linkage, not arithmetic.
- `outstandingAmount` = `max(0, netPayable − settledToDate)`, floored at zero.
- `excessAmount` = `settledToDate − netPayable` (only when over-settled).

And a state machine (all comparisons within ±0.01):

```
not_settled        (settledToDate ~ 0)
   |  record a partial transfer
   v
partially_settled  (0 < settledToDate < netPayable)
   |  settle the rest
   v
fully_settled      (settledToDate ~ netPayable)   ← normal end state

over_settled       (settledToDate > netPayable)    ← needs super_admin override
```

A fully-reversed ledger reverts to `not_settled`.

Stage 5 Recording a Transfer

When the admin has actually wired the money and records it, the step order *is* the design (there is no DB transaction, so ordering plus a unique index makes every failure land somewhere safe):

- Resolve the listing (404 if absent).
- Require an acting admin (the recording admin is mandatory on every row).
- **Idempotency pre-read** before validation and before any write — a double-click returns the already-recorded ledger, leaving no spurious audit row.
- Validate: amount a positive whole rupee, reference (UTR) present, date not in the future.
- **Over-settlement guard** — rejected unless a super admin supplies an override + reason.
- **Audit write first, allowed to throw** — no settlement may exist without its audit record.
- Insert; a duplicate-key race is answered by returning the winner.
- Refold the ledger, notify the owner (best effort), return the updated figures.

A correction is **never a mutation** — a reversal is a second, negative mirror row. You cannot reverse a reversal, nor reverse the same entry twice.

Stage 6 Two Views: Admin vs Owner

- **Admin view** — full breakdown including internal fields (notes, override reason, recording admin, reversal linkage).
- **Owner view** — a whitelist projection: only owner-side money, with ledger rows stripped to amount, reference, date, and a reversed flag. A field added later cannot accidentally leak to owners.

Stage 7 Routes & Access Control

The three settlement routes are the platform's **only money-write routes**:

- **Read settlement** — standard role guard (admin / super admin).
- **Record entry** — strict guard (role must be exactly admin / super admin) plus a schema that strips unknown keys, so a caller cannot smuggle internal fields into the row.
- **Reverse entry** — same strict guard.

Stage 8 The Admin Panel

The panel enforces exactly one of `loading / ready / empty / error`, so a fetch **failure can never be mistaken for a zero balance**. It shows three headline cards (Net Payable, Settled To Date, Outstanding), two groups (Collected from buyers, Owed to owner), refunds separately, and the message **"Payout not initiated — nothing to settle yet"** whenever `netPayable ≤ 0`. That message is the designed state when no payout exists.

Per-field provenance

Field	Source
grossCollected	Sum of Payment.totalAmount (success)
platformFeeCollected	Sum of Payment.platformFee (success)
gstRetained	Sum of Payment.gstAmount (success)
refundedTotal	Sum of Payment.totalAmount (refunded)
ownerGross	Sum of Payout.grossAmount
platformCommission	Sum of Payout.platformCommission
netPayable	Sum of Payout.netAmount
settledToDate	Derived: sum of effective Settlement.settledAmount
outstandingAmount	Derived: max(0, netPayable − settledToDate)
excessAmount	Derived: settledToDate − netPayable (if over-settled)
state	Derived lattice (not / partial / full / over settled)

> **Practical takeaway.** If the owner-side figures read ₹0.00 while buyers' money was collected, the chain is simply broken at Stage 2: payments exist, but no payout has been raised. To populate "Owed to Owner," a payout must be raised for the listing (with a valid gross amount and the owner's bank details on file). Once the payout exists, the owner side fills in, and the admin can record settlement entries against it as the money is actually transferred.